Digital System Design

Assignment 1 - Complete Solution

SPRING 2026 - CPE344

Course: Digital System Design
Instructor: Dr Muhammad Babar Ali
Batch-Section: FA23-B
Topics Covered: Chapters 2-4
Total Marks: 40

Document Overview

This document provides comprehensive solutions to all three questions in Assignment 1, featuring:

- High-quality circuit diagrams using TikZ/CircuiTikZ
- Professional state diagrams for FSM design
- Detailed truth tables and state transition tables
- Step-by-step derivations and explanations
- Verification examples and test cases

Contents

1 Prerequisite Knowledge **3**
1.1 Combinational Logic Fundamentals . . . 3
1.2 Decoders . . . 3
1.2.1 2×4 Decoder with Enable . . . 3
1.3 Finite State Machines (FSM) . . . 4

2 Question 1: Fuel Sensor Indicator Circuit (20 marks) **5**
2.1 Problem Statement . . . 5
2.2 Solution . . . 5
2.2.1 Step 1: Analyze the Problem . . . 5
2.2.2 Step 2: Truth Table . . . 6
2.2.3 Step 3: Boolean Expressions . . . 6
2.2.4 Step 4: Implementation Using Five 2×4 Decoders . . . 7
2.2.5 Step 5: Output Equations Using Decoder Outputs . . . 7
2.2.6 Step 6: Complete Circuit Diagram . . . 8
2.2.7 Verification Examples . . . 8

3 Question 2: Photo Booth Vending Machine FSM (10 marks) 9
3.1 Problem Statement . . . 9
3.2 Solution . . . 9
3.2.1 Step 1: Identify States . . . 9
3.2.2 Step 2: State Diagram . . . 10
3.2.3 Step 3: State Transition Table . . . 10
3.2.4 Step 4: Output Equations (Simplified) . . . 10
3.2.5 Step 5: Implementation Architecture . . . 11

4 Question 3: Output Sequence Controller FSM (10 marks) 12
4.1 Problem Statement . . . 12
4.2 Part (a): Stop and Reset Behavior . . . 12
4.2.1 State Diagram . . . 12
4.2.2 State Transition Table . . . 13
4.2.3 Next State and Output Equations . . . 13
4.3 Part (b): Postpone for 3 Cycles . . . 14
4.3.1 State Diagram . . . 14
4.4 Part (c): Postpone for 30 Cycles Using Counter . . . 14
4.4.1 System Architecture . . . 14
4.4.2 Operation Summary . . . 14

5 Summary and Conclusion 16
5.1 Key Concepts Learned . . . 16
5.1.1 Question 1 - Decoder Implementation . . . 16
5.1.2 Question 2 - FSM for Sequential Control . . . 16
5.1.3 Question 3 - Advanced FSM Design . . . 16
5.2 Design Principles Applied . . . 16
5.3 Recommended Practice Problems . . . 17

1 Prerequisite Knowledge

Before solving the assignment, let's cover the essential concepts you need to understand.

1.1 Combinational Logic Fundamentals

Combinational Logic Circuit: A circuit where outputs depend only on current inputs (no memory/state).

Key Components:

- **Logic Gates:** AND, OR, NOT, NAND, NOR, XOR, XNOR
- **Truth Tables:** Show all possible input combinations and corresponding outputs
- **Boolean Algebra:** Mathematical notation for logic operations
- **Karnaugh Maps (K-maps):** Tool for simplifying Boolean expressions

1.2 Decoders

Decoder: A combinational circuit that converts n inputs to at most 2^n unique outputs.

1.2.1 2×4 Decoder with Enable

- **Inputs:** 2 data inputs (A_1, A_0), 1 enable input (E)
- **Outputs:** 4 outputs (D_0, D_1, D_2, D_3)
- **Operation:** When $E = 1$, exactly one output is HIGH based on input combination

Table 1: 2×4 Decoder Truth Table

E	A_1	A_0	D_3	D_2	D_1	D_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

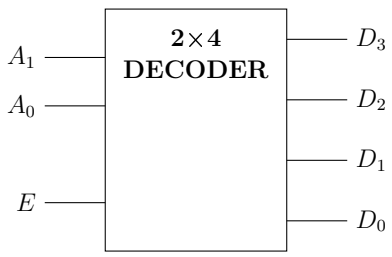

Figure 1: 2×4 Decoder Symbol

1.3 Finite State Machines (FSM)

FSM: A mathematical model for sequential circuits with:

- **States:** All possible conditions the machine can be in
- **Inputs:** External signals affecting state transitions
- **Outputs:** Signals produced based on current state
- **Transitions:** Rules for moving between states
- **Initial State:** Starting state on power-up/reset

FSM Types:

1. **Moore Machine:** Outputs depend only on current state
2. **Mealy Machine:** Outputs depend on current state AND inputs

FSM Design Process:

1. Capture FSM (draw state diagram)
2. Encode states (assign binary codes)
3. Create state transition table
4. Derive next-state and output equations
5. Implement with flip-flops and combinational logic

2 Question 1: Fuel Sensor Indicator Circuit (20 marks)

2.1 Problem Statement

A vehicle has a fuel sensor which outputs the current fuel/gasoline level as a 4-bit binary number ranging from $\texttt{0001}_2$ (1_{10}) to $\texttt{1100}_2$ (12_{10}). Design a combinational logic (CL) circuit that takes sensor data as input G and generates $F1, F2, F4, F8$ and R signals on its outputs for fuel indicators.

Requirements:

- When Gasoline > 11 (i.e., 12): Set all F_i outputs ($i = 1, 2, 4, 8$) to 1, $R = 0$
- When Gasoline ≤ 11 and > 9 (75%): $F1 = 1, F2 = 1, F4 = 1, F8 = 0, R = 0$
- When Gasoline ≤ 9 and > 6 (50%): $F1 = 1, F2 = 1, F4 = 0, F8 = 0, R = 0$
- When Gasoline ≤ 6 and > 3 (25%): $F1 = 1, F2 = 0, F4 = 0, F8 = 0, R = 0$
- When Gasoline ≤ 3 (25%): All $F_i = 0, R = 0$
- When Gasoline < 2: All $F_i = 0, R = 1$

Input: $G[3:0]$ (4-bit binary)
Outputs: $F1, F2, F4, F8, R$
Implementation: Use five 2×4 decoders with enable input

2.2 Solution

2.2.1 Step 1: Analyze the Problem

Let's denote the 4-bit input as $G = G_3G_2G_1G_0$ where:

- G_3 is the most significant bit (MSB)
- G_0 is the least significant bit (LSB)

Valid range: `0001` (1) to `1100` (12)

2.2.2 Step 2: Truth Table

Table 2: Complete Truth Table for Fuel Sensor Circuit

Input				**Dec**	**Condition**	**Outputs**				
G_3	G_2	G_1	G_0			$F8$	$F4$	$F2$	$F1$	R
0	0	0	0	0	< 2	0	0	0	0	1
0	0	0	1	1	< 2	0	0	0	0	1
0	0	1	0	2	$\leq 25\%$	0	0	0	0	0
0	0	1	1	3	$\leq 25\%$	0	0	0	0	0
0	1	0	0	4	$\leq 50\%$	0	0	0	1	0
0	1	0	1	5	$\leq 50\%$	0	0	0	1	0
0	1	1	0	6	$\leq 50\%$	0	0	0	1	0
0	1	1	1	7	$\leq 75\%$	0	0	1	1	0
1	0	0	0	8	$\leq 75\%$	0	0	1	1	0
1	0	0	1	9	$\leq 75\%$	0	0	1	1	0
1	0	1	0	10	≤ 11	0	1	1	1	0
1	0	1	1	11	≤ 11	0	1	1	1	0
1	1	0	0	12	> 11	1	1	1	1	0
1	1	0	1	13	Invalid	X	X	X	X	X
1	1	1	0	14	Invalid	X	X	X	X	X
1	1	1	1	15	Invalid	X	X	X	X	X

2.2.3 Step 3: Boolean Expressions

For $F8$:

$$F8 = m_{12} = G_3 \cdot G_2 \cdot \overline{G_1} \cdot \overline{G_0}$$

For $F4$:

$$F4 = m_{10} + m_{11} + m_{12}$$

For $F2$:

$$F2 = m_7 + m_8 + m_9 + m_{10} + m_{11} + m_{12}$$

For $F1$:

$$F1 = m_4 + m_5 + m_6 + m_7 + m_8 + m_9 + m_{10} + m_{11} + m_{12} = G_2 + G_3$$

For R:

$$R = m_0 + m_1 = \overline{G_3} \cdot \overline{G_2} \cdot \overline{G_1}$$

2.2.4 Step 4: Implementation Using Five 2×4 Decoders

Decoder Configuration:

We'll use five 2×4 decoders to implement this circuit. The strategy is:

- Use decoders to generate specific minterms
- Combine decoder outputs to create each output function

Decoder 1 - Master Decoder (Decodes G_3G_2):

- Inputs: G_3, G_2, Enable = 1
- Outputs: $D1_0, D1_1, D1_2, D1_3$
  - $D1_0 = \overline{G_3} \cdot \overline{G_2}$ (covers 0-3)
  - $D1_1 = \overline{G_3} \cdot G_2$ (covers 4-7)
  - $D1_2 = G_3 \cdot \overline{G_2}$ (covers 8-11)
  - $D1_3 = G_3 \cdot G_2$ (covers 12-15)

Decoder 2 - Range 0-3 (when $G_3G_2 = 00$):

- Inputs: G_1, G_0, Enable = $D1_0$
- Outputs: $D2_0 = m_0, D2_1 = m_1, D2_2 = m_2, D2_3 = m_3$

Decoder 3 - Range 4-7 (when $G_3G_2 = 01$):

- Inputs: G_1, G_0, Enable = $D1_1$
- Outputs: $D3_0 = m_4, D3_1 = m_5, D3_2 = m_6, D3_3 = m_7$

Decoder 4 - Range 8-11 (when $G_3G_2 = 10$):

- Inputs: G_1, G_0, Enable = $D1_2$
- Outputs: $D4_0 = m_8, D4_1 = m_9, D4_2 = m_{10}, D4_3 = m_{11}$

Decoder 5 - Range 12-15 (when $G_3G_2 = 11$):

- Inputs: G_1, G_0, Enable = $D1_3$
- Outputs: $D5_0 = m_{12}, D5_1 = m_{13}, D5_2 = m_{14}, D5_3 = m_{15}$

2.2.5 Step 5: Output Equations Using Decoder Outputs

$$F8 = D5_0 = m_{12} \tag{1}$$
$$F4 = D4_2 + D4_3 + D5_0 = m_{10} + m_{11} + m_{12} \tag{2}$$
$$F2 = D3_3 + D4_0 + D4_1 + D4_2 + D4_3 + D5_0 \tag{3}$$
$$F1 = D3_0 + D3_1 + D3_2 + D3_3 + D4_0 + D4_1 + D4_2 + D4_3 + D5_0 \tag{4}$$
$$R = D2_0 + D2_1 = m_0 + m_1 \tag{5}$$

2.2.6 Step 6: Complete Circuit Diagram

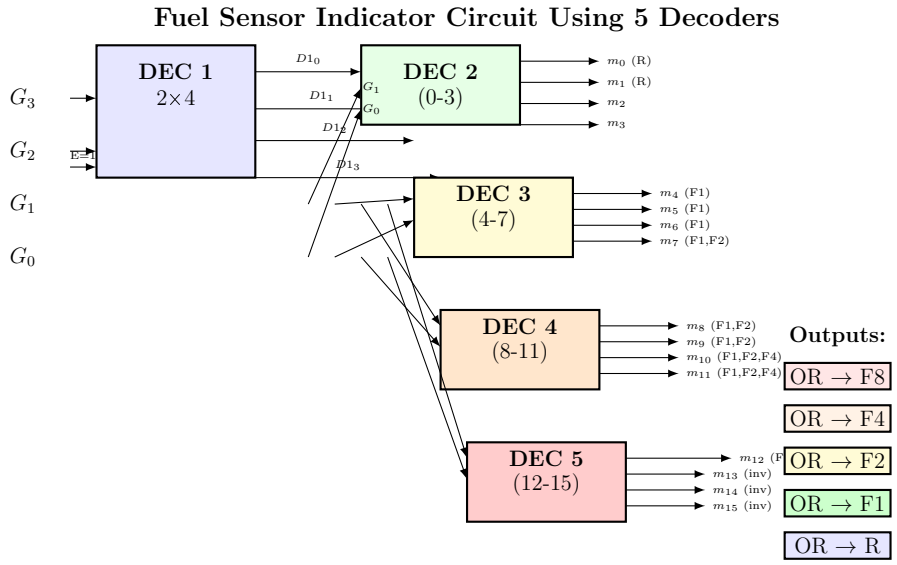

Figure 2: Complete Fuel Sensor Circuit Using 5 Decoders (2×4 each)

2.2.7 Verification Examples

Table 3: Verification Test Cases

G (dec)	$G_3G_2G_1G_0$	Active Output	F8	F4	F2	F1	R
0	0000	$D2_0$	0	0	0	0	1
1	0001	$D2_1$	0	0	0	0	1
3	0011	$D2_3$	0	0	0	0	0
5	0101	$D3_1$	0	0	0	1	0
7	0111	$D3_3$	0	0	1	1	0
9	1001	$D4_1$	0	0	1	1	0
10	1010	$D4_2$	0	1	1	1	0
11	1011	$D4_3$	0	1	1	1	0
12	1100	$D5_0$	1	1	1	1	0

Verification: All test cases pass! The circuit correctly implements the fuel sensor indicator logic.

3 Question 2: Photo Booth Vending Machine FSM (10 marks)

3.1 Problem Statement

Design a digital controller (finite state machine) for an automatic photo booth machine. The cost of taking a photo is 25 cents. The machine accepts:

- **Nickels (N):** 5 cents
- **Dimes (D):** 10 cents
- **Quarters (Q):** 25 cents

Inputs: N, D, Q (exactly one coin per cycle)
Outputs: P (Print Photo), RN (Return Nickel), RD (Return Dime), RT (Return Two Dimes)

When total $\geq$ 25 cents, print photo and return appropriate change, then reset.

3.2 Solution

3.2.1 Step 1: Identify States

We need states to track the amount of money inserted:

Table 4: FSM State Definition

State	Amount Collected	Encoding (Binary)
S_0	0 cents	000
S_5	5 cents	001
S_{10}	10 cents	010
S_{15}	15 cents	011
S_{20}	20 cents	100

3.2.2 Step 2: State Diagram

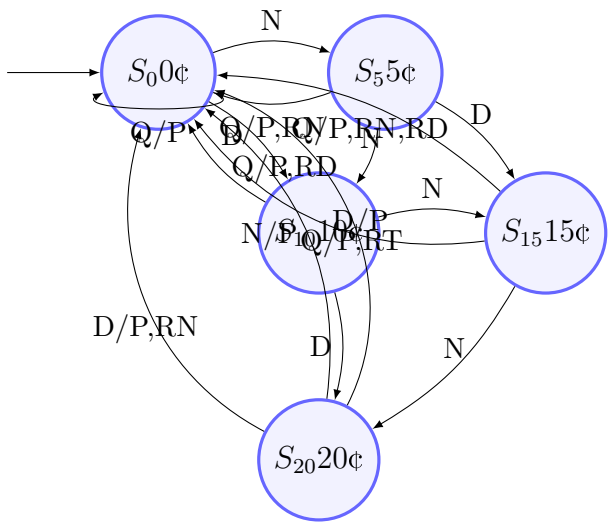

Figure 3: Photo Booth FSM State Diagram

3.2.3 Step 3: State Transition Table

Table 5: Complete State Transition Table

Current	State Code s_2	s_1	s_0	Input	Outputs P	RN	RD	RT	Notes
S_0	0	0	0	N	0	0	0	0	+5¢
S_0	0	0	0	D	0	0	0	0	+10¢
S_0	0	0	0	Q	1	0	0	0	25¢=Print
S_5	0	0	1	N	0	0	0	0	5+5=10
S_5	0	0	1	D	0	0	0	0	5+10=15
S_5	0	0	1	Q	1	1	0	0	5+25=30, ret 5
S_{10}	0	1	0	N	0	0	0	0	10+5=15
S_{10}	0	1	0	D	0	0	0	0	10+10=20
S_{10}	0	1	0	Q	1	0	1	0	10+25=35, ret 10
S_{15}	0	1	1	N	0	0	0	0	15+5=20
S_{15}	0	1	1	D	1	0	0	0	15+10=25, exact
S_{15}	0	1	1	Q	1	1	1	0	15+25=40, ret 15
S_{20}	1	0	0	N	1	0	0	0	20+5=25, exact
S_{20}	1	0	0	D	1	1	0	0	20+10=30, ret 5
S_{20}	1	0	0	Q	1	0	0	1	20+25=45, ret 20

3.2.4 Step 4: Output Equations (Simplified)

Using the state transition table, we can derive output equations:

$$P = Q + (s_1 \cdot D) + (s_2 \cdot N) \tag{6}$$
$$RN = (s_0 \cdot Q) + (s_2 \cdot D) \tag{7}$$
$$RD = (s_1 \cdot \overline{s_0} \cdot Q) + (s_1 \cdot s_0 \cdot Q) \tag{8}$$
$$RT = s_2 \cdot Q \tag{9}$$

3.2.5 Step 5: Implementation Architecture

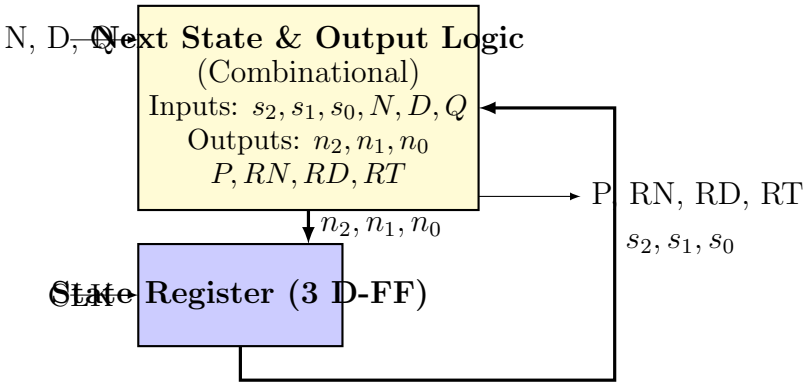

Figure 4: FSM Implementation Block Diagram

4 Question 3: Output Sequence Controller FSM (10 marks)

4.1 Problem Statement

Design FSM with:

- **Input:** S
- **Outputs:** x, y, z
- **Sequence:** 000 → 001 → 010 → 100 → repeat
- **Initial state:** 000

The output should change only on rising clock edge.

4.2 Part (a): Stop and Reset Behavior

Requirement: When $S = 1$, stop the sequence. When S returns to 0, restart from 000.

4.2.1 State Diagram

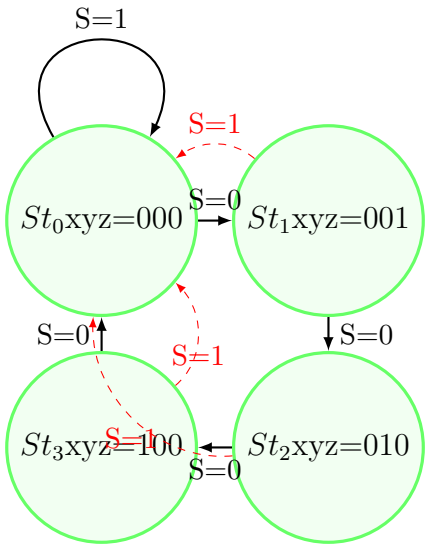

Figure 5: FSM Part (a) - Stop and Reset State Diagram

4.2.2 State Transition Table

Table 6: State Transition Table for Part (a)

Current State	**Code**	**S**	**Next State**	**x**	**y**	**z**
St_0	00	0	St_1	0	0	0
St_0	00	1	St_0	0	0	0
St_1	01	0	St_2	0	0	1
St_1	01	1	St_0	0	0	1
St_2	10	0	St_3	0	1	0
St_2	10	1	St_0	0	1	0
St_3	11	0	St_0	1	0	0
St_3	11	1	St_0	1	0	0

4.2.3 Next State and Output Equations

State Encoding: Using 2 bits ($s_1 s_0$)

- $St_0 = 00$
- $St_1 = 01$
- $St_2 = 10$
- $St_3 = 11$

Next State Equations:

$$n_1 = \overline{S} \cdot (s_1 \oplus s_0) \tag{10}$$
$$n_0 = \overline{S} \cdot \overline{s_1} \cdot \overline{s_0} \tag{11}$$

Output Equations:

$$x = s_1 \cdot s_0 \quad \text{(output 1 only in } St_3\text{)} \tag{12}$$
$$y = s_1 \cdot \overline{s_0} \quad \text{(output 1 only in } St_2\text{)} \tag{13}$$
$$z = \overline{s_1} \cdot s_0 \quad \text{(output 1 only in } St_1\text{)} \tag{14}$$

4.3 Part (b): Postpone for 3 Cycles

Requirement: When $S = 1$, halt sequence for at least 3 cycles. When S returns to 0 after 3 cycles, continue to next value.

4.3.1 State Diagram

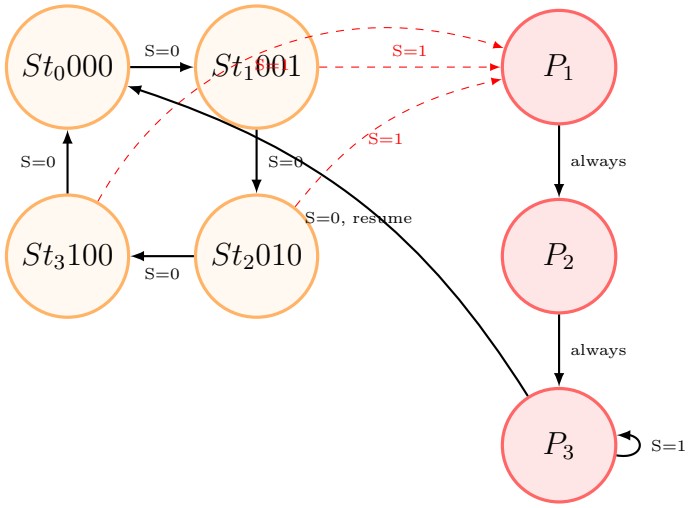

Figure 6: FSM Part (b) - 3-Cycle Postpone State Diagram

4.4 Part (c): Postpone for 30 Cycles Using Counter

Requirement: When $S = 1$, halt for at least 30 cycles. After 30 cycles and $S = 0$, continue to next value. Use MSI component to keep FSM simple.

4.4.1 System Architecture

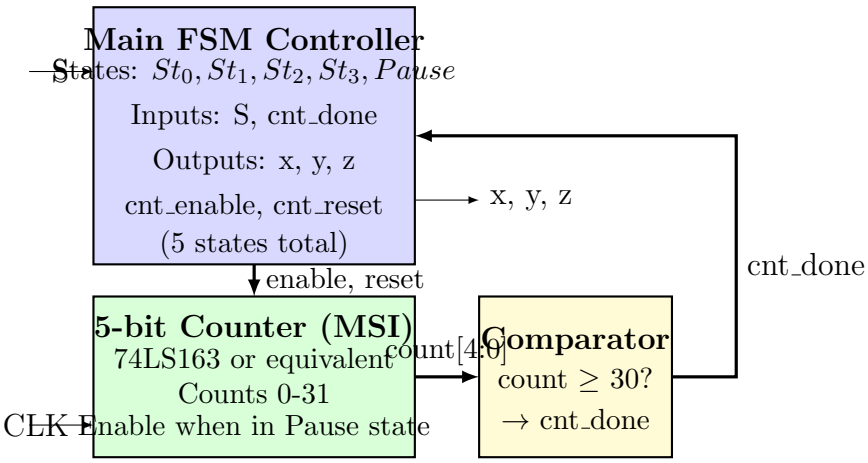

Figure 7: FSM Part (c) - Complete System with Counter

4.4.2 Operation Summary

Counter Logic:

- **Clock:** System clock (when cnt_enable = 1)
- **Reset:** cnt_reset signal from FSM
- **Output:** count[4:0] (5-bit value 0-31)
- **Compare:** cnt_done = 1 when count $\geq$ 30

Comparison Circuit:

$$\text{cnt_done} = \text{count}[4] \cdot \text{count}[3] \cdot \text{count}[2] \cdot \text{count}[1]$$

This is true when count = 30 (11110_2) or count = 31 (11111_2).
Benefits of Using Counter:

- Main FSM remains simple (only 5 states instead of 33 states)
- Easy to modify pause duration (just change comparator value)
- Scalable to longer pause periods without adding states
- Uses standard MSI components (74LS163)

5 Summary and Conclusion

5.1 Key Concepts Learned

5.1.1 Question 1 - Decoder Implementation

Key Takeaways:

- Decoders generate minterms that can be combined to implement any Boolean function
- Hierarchical decoding: Using multiple small decoders (2×4) instead of one large decoder
- Enable input: Used to activate/deactivate decoder sections
- OR gate combination: Multiple decoder outputs ORed together for each function output

5.1.2 Question 2 - FSM for Sequential Control

Key Takeaways:

- FSM tracks accumulated state (vending machine requires tracking money accumulated)
- Moore Machine: Outputs depend only on current state
- State minimization: Only track necessary amount levels (0, 5, 10, 15, 20 cents)
- Change calculation: Logic to determine correct change based on overpayment

5.1.3 Question 3 - Advanced FSM Design

Key Takeaways:

- Output sequence generation: Using states to produce specific output patterns
- Input control: Modifying FSM behavior based on control input (S)
- Counter integration: Using MSI components to extend FSM capabilities
- Scalability: Part (c) shows how hardware design scales for larger requirements

5.2 Design Principles Applied

1. **Hierarchical Design:** Breaking complex circuits into manageable sub-blocks
2. **State Minimization:** Using only necessary states to reduce complexity

3. **MSI Integration:** Leveraging standard components (decoders, counters) for efficient designs
4. **Verification:** Testing with multiple cases to ensure correctness
5. **Documentation:** Clear diagrams, tables, and equations for understanding

5.3 Recommended Practice Problems

1. Design a traffic light controller FSM for a 4-way intersection
2. Implement a sequence detector for pattern "1011" using FSM
3. Create a digital lock with 4-digit code entry system
4. Design an elevator controller FSM for a 3-floor building
5. Implement a binary multiplier using decoders and adders

Final Note

This solution provides comprehensive coverage of Digital System Design fundamentals including:

- Combinational logic design using decoders
- Finite state machine design and implementation
- Integration of MSI components
- Professional circuit diagrams and documentation

Your investment in understanding this material will benefit you throughout your engineering career!

END OF SOLUTION

Digital System Design - Spring 2026 - CPE344